



3.1. Finding All Paths in a Graph

"From Source to Sink, Exploring Every Link."

Main Section:

3. Backtracking

Contents:

[1. Problem Definition](#)

[2. The Workflow](#)

[Summary](#)

[References](#)

Now that we understand how Backtracking works, let's apply it to one of the most fundamental problems in graph theory: ***Finding all possible paths between two vertices.***

This problem is simple enough to introduce the core mechanics of Backtracking, yet powerful enough to reveal the true idea behind recursive exploration. Unlike shortest-path algorithms that try to optimize distance, this problem has a different objective: ***Explore every valid possibility.***

Given a directed graph, we begin at a starting node, follow the direction of the edges, and search for every possible path that eventually reaches the destination node.

However, there is an important restriction: ***A valid path must not contain cycles.*** In other words, once a node has already appeared in the current path, we cannot revisit it again within the same traversal.

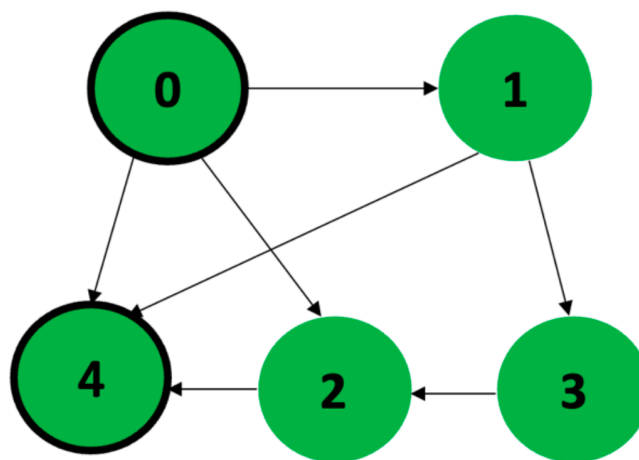
At first, this may sound similar to brute force. And technically, it is a form of exhaustive exploration. But Backtracking adds something important, it keeps

track of the current path, abandons invalid paths early, and systematically returns to previous decisions whenever exploration is complete.

This creates a recursive search process where the algorithm continuously explores deeper paths, records successful solutions, and backtracks to discover unexplored alternatives. And this exact behavior becomes the foundation of almost every Backtracking problem we will study later.

1. Problem Definition

Given a directed graph, the objective is to find **all possible paths** between two vertices.



These paths must follow the direction of the edges, must begin from the starting node, and must not contain cycles.

The problem may sound straightforward at first. But once a graph contains multiple branches and connections, the number of possible paths can grow very quickly.

So how does Backtracking solve this? The algorithm begins from the starting node and recursively explores every adjacent node connected to it.

For every recursive step: The current node is added into the current path, the algorithm checks whether the destination node has been reached, and if not, it continues exploring deeper.

Eventually, one of two things happens:

- The current node becomes the destination node → The current path is recorded as a valid solution.
- The current path can no longer continue → The algorithm backtracks to the previous node and explores another branch.

This process repeats recursively until every possible path has been explored, and every valid solution has been recorded.

The most important part is this: ***Backtracking does not search for just one answer.*** It systematically searches for **all possible valid answers**.

To avoid cycles, the algorithm also maintains a structure (usually a visited list or current path list) that tracks which nodes already exist in the current traversal. This ensures the algorithm never revisits the same node within the same path.

2. The Workflow

Suppose we want to find all possible paths from: starting node **0**, to destination node **4**. To avoid cycles, we maintain a structure that keeps track of the current path.

- Initially: Current path = $[0]$

From node **0**, there are three adjacent nodes: node **4**, node **2**, node **1**. The algorithm now begins exploring recursively.

- **Step 1 — Move from Node 0 → Node 4**

Current path becomes: $[0, 4]$

Since node **4** is the destination node: the path is recorded. So we obtain:

$$[0, 4]$$

This is our first valid path.

- **Step 2 — Backtrack to Node 0**

After recording the path, node **4** is removed from the current path, and the algorithm returns to node **0**. Current path becomes: $[0]$.

The algorithm now continues exploring unexplored adjacent nodes.

- **Step 3 — Move from Node 0 → Node 2**

Current path becomes: $[0, 2]$. From node **2**, there is one adjacent node: node **4**

- **Step 4 — Move from Node 2 → Node 4**

Current path becomes: $[0, 2, 4]$. Again, node **4** is the destination node.

So the path is recorded:

$$[0, 2, 4]$$

- **Step 5 — Backtrack to Node 0**

Since node **2** has no more unexplored adjacent nodes: node **4** is removed, node **2** is removed, and the algorithm returns back to node **0**. Current path becomes: $[0]$

- **Step 6 — Move from Node 0 → Node 1**

Current path becomes: $[0, 1]$. From node **1**, there are two adjacent nodes: node **4** and node **3**

- **Step 7 — Move from Node 1 → Node 4**

Current path becomes: $[0, 1, 4]$. Since node **4** is the destination, the path is recorded.

So we obtain:

$$[0, 1, 4]$$

- **Step 8 — Backtrack to Node 1**

Node **4** is removed from the path. Current path becomes: $[0, 1]$

The algorithm now explores the remaining adjacent node.

- **Step 9 — Move from Node 1 → Node 3**

Current path becomes: $[0, 1, 3]$. From node **3**, there is one adjacent node node **2**

- **Step 10 — Move from Node 3 → Node 2**

Current path becomes: $[0, 1, 3, 2]$. From node **2**, there is one adjacent node: node **4**

- **Step 11 — Move from Node 2 → Node 4**

Current path becomes: $[0, 1, 3, 2, 4]$. Since node **4** is the destination node, the path is recorded.

So we obtain:

$$[0, 1, 3, 2, 4]$$

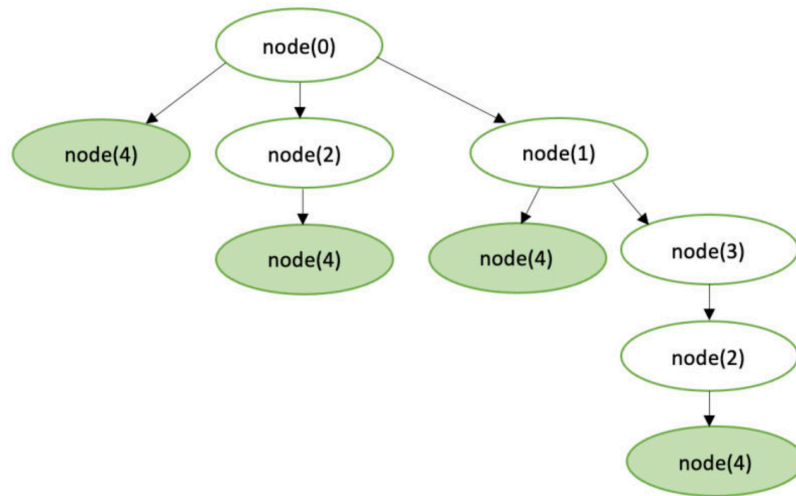
At this point, all possible paths from node **0** to node **4** have been explored.

The discovered paths are:

$$[0, 4]; \quad [0, 2, 4]; \quad [0, 1, 4]; \quad [0, 1, 3, 2, 4]$$

The State Space Tree

If we visualize this recursive exploration process, we obtain a structure called the ***State Space Tree***.



This tree represents every decision made by the algorithm, every recursive branch explored, and every possible state reached during the search.

Almost every Backtracking problem can eventually be represented using this exact structure. The state space tree is one of the most important concepts in Backtracking because it visually demonstrates exploration, recursion, branching, and backtracking itself.

Each branch represents: ***“What happens if we choose this option next?”***. And whenever a branch fails or finishes, the algorithm climbs back upward through the tree, and continues exploring another branch.

That recursive exploration of the state space tree is the true heart of Backtracking.

Summary

Observe something interesting here. Even though we are solving this problem using Backtracking, the actual traversal behavior looks extremely familiar. The algorithm continuously moves deeper into one branch before returning backward and exploring another branch. In other words, we are essentially performing a Depth-First Search (DFS).

The difference is that Backtracking adds additional control over the exploration process. Instead of simply traversing nodes, it also keeps track of the current path, avoids cycles, records valid solutions, and carefully undoes decisions

when returning to previous states. You can think of Backtracking as DFS combined with constraint checking and state management.

This problem is extremely common in areas involving graph traversal and path exploration. Examples include finding routes in navigation systems, network routing, puzzle solving, game pathfinding, dependency analysis, workflow exploration, and state-space searching in Artificial Intelligence.

And more importantly, this problem introduces one of the biggest ideas in Backtracking:

The algorithm explores an entire decision tree recursively while continuously undoing and retrying choices until every valid solution has been discovered.

That recursive exploration process becomes the foundation for almost every Backtracking problem that follows.

References

<https://www.geeksforgeeks.org/dsa/count-possible-paths-two-vertices/>

<https://www.baeldung.com/cs/simple-paths-between-two-vertices>